Bitácora proyecto corto 1

David Araya Mora

Transmisor:

Módulo de codificación de paridad par:

Se espera un módulo de 4 entradas y 7 salidas codificadas en Hamming (7,4)

Se define:

$i[3:0]$ = Palabra de 4 bits

$c[2:0]$ = 3 bits de paridad

Se usa el operador XOR, para verificar la paridad:

Ecuaciones boleanas

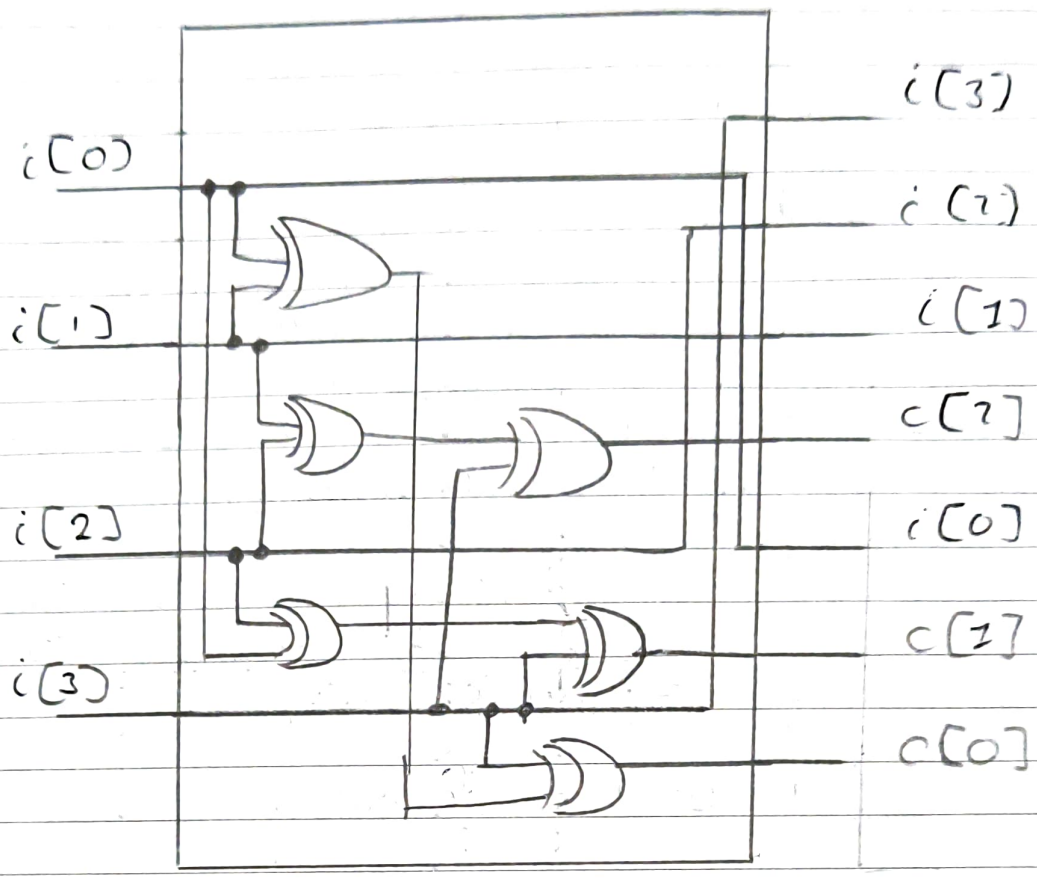
$$C_1 = i_0 \oplus i_1 \oplus i_3$$

$$C_2 = i_0 \oplus i_2 \oplus i_3$$

$$C_3 = i_1 \oplus i_2 \oplus i_3$$

Se obtiene una palabra de 7 bits:

$$\{i_3, i_2, i_1, C_2, i_0, C_1, C_0\}$$



Módulo de codificador (7,4)

En el testbench se obtuvieron los siguientes resultados:

Testbench:

i_3	i_2	i_1	i_0	C_0	C_1	C_2
0	0	0	0	0	0	0
0	0	0	1	1	1	0
0	0	1	0	1	0	1
0	0	1	1	0	1	1
0	1	0	0	0	1	1
0	1	0	1	1	0	1
0	1	1	0	1	1	0
0	1	1	1	0	0	0
1	0	0	0	1	1	1
1	0	0	1	0	0	1
1	0	1	0	0	1	0
1	0	1	1	1	0	0
1	1	0	0	1	0	0
1	1	0	1	0	1	0
1	1	1	0	0	0	1
1	1	1	1	1	1	1

Funciona correctamente en físico, no hay errores por reportar.

Módulo de modificación binario a 7 seg.

Se espera un módulo con 4 entradas y 7 salidas, que describa un 7-seg display.

i_3	i_2	i_1	i_0	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1
1	0	1	0	1	1	1	0	1	1	1
1	0	1	1	0	0	1	1	1	1	1
1	1	0	0	1	0	0	1	1	1	0
1	1	0	1	0	1	1	1	1	0	1
1	1	1	0	1	0	0	1	1	1	1
1	1	1	1	1	0	0	0	1	1	1

Se pueden generar simplificaciones de Karnaugh para cada uno de los 7 segmentos, así:

$$a = \overline{i_3} i_2 i_0 + \overline{i_3} i_1 + i_3 \overline{i_2} \overline{i_1} + i_3 \overline{i_0}$$

$$+ \overline{i_2} \overline{i_0} + i_2 i_1$$

$$b = \overline{i_2}\,\overline{i_1} + \overline{i_2}\,\overline{i_0} + \overline{i_3}\,\overline{i_1}\,\overline{i_0} + \overline{i_3}\,i_1 i_0 + i_3\,\overline{i_1}\,i_0$$

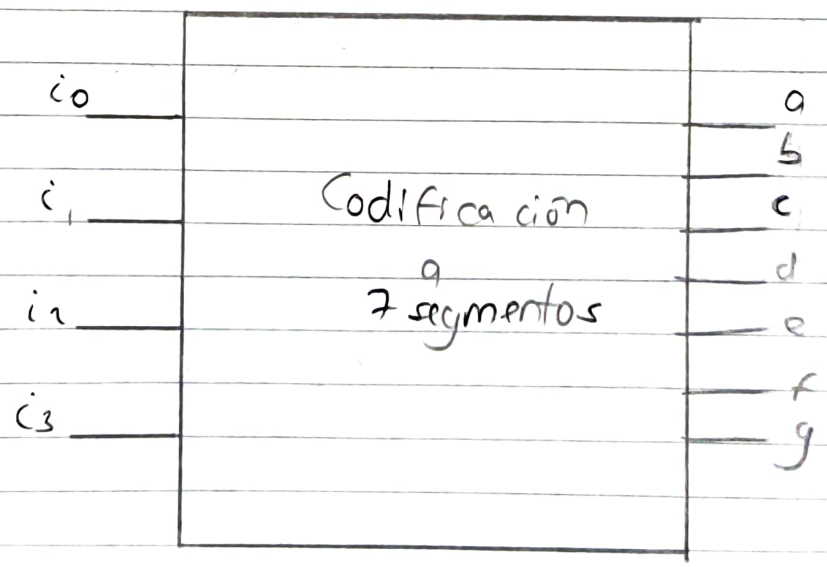
$$c = \overline{i_2}\,\overline{i_1} + \overline{i_2}\,i_0 + \overline{i_1}\,i_0 + \overline{i_3}\,i_2 + i_3\,\overline{i_2}$$

$$d = \overline{i_3}\,i_1\,\overline{i_0} + i_3\,\overline{i_1} + \overline{i_2}\,\overline{i_1}\,\overline{i_0} + \overline{i_2}\,i_1 i_0$$
$$+ i_2\,\overline{i_1}\,i_0 + i_2\,i_1\,\overline{i_0}$$

$$e = i_3 i_2 + i_3 i_1 + \overline{i_2}\,\overline{i_0} + i_1\,\overline{i_0}$$

$$f = \overline{i_3}\,i_2\,\overline{i_1} + i_3\,\overline{i_2} + i_3 i_1 + i_2\,\overline{i_0} + \overline{i_1}\,\overline{i_0}$$

$$g = \overline{i_3}\,i_2\,\overline{i_1} + i_3\,\overline{i_2} + i_3 i_0 + \overline{i_2}\,i_1 + i_1\,\overline{i_0}$$

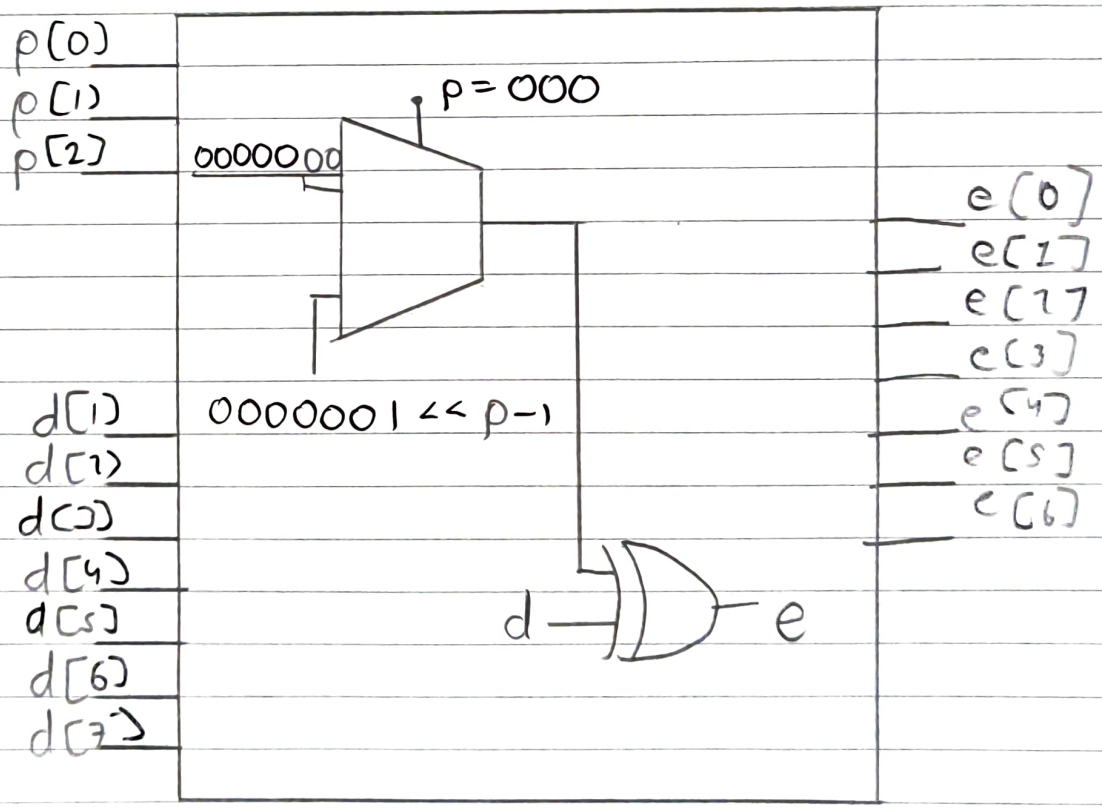


El testbench resultó igual que la tabla de verdad anterior.

Módulo generador de error

Se espera un módulo de 10 entradas y 7 salidas donde uno de los bits se invierta.

- 7 bits de datos sin error
- 3 bits para escoger la posición del error
- 7 bits de salida de data con error.



a p se le resta uno, ya que en la posición 000, se espera que no se presente error.

Resultados del testbench:

Asúmase una palabra "1111111":

p_2	p_1	p_0	d_6	d_5	d_4	d_3	d_2	d_1	d_0
0	0	0	1	1	1	1	1	1	1
0	0	1	1	1	1	1	1	1	0
0	1	0	1	1	1	1	1	0	1
0	1	1	1	1	1	1	0	1	1
1	0	0	1	1	1	0	1	1	1
1	0	1	1	1	0	1	1	1	1
1	1	0	1	0	1	1	1	1	1
1	1	1	0	1	1	1	1	1	1